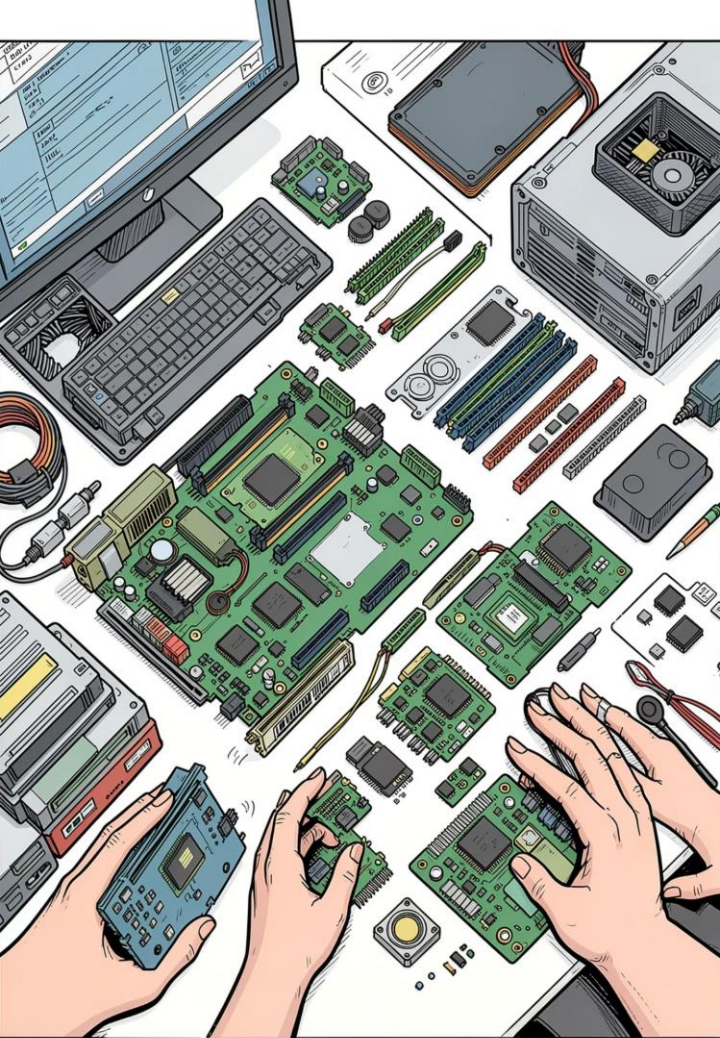




Patrón de Diseño Builder

Programación Orientada a Objetos

ESTUDIANTE: ANCHUNDIA LUCAS JOSÉ ANDRÉS · NIVEL: 3A

DOCENTE: EDGARDO PANCHANA

¿Por qué usar patrones de diseño?

Los patrones de diseño son **soluciones reutilizables** a problemas recurrentes en el desarrollo de software. No reinventan la rueda: capturan la experiencia colectiva de la comunidad para escribir código más limpio, mantenible y escalable.

Organización

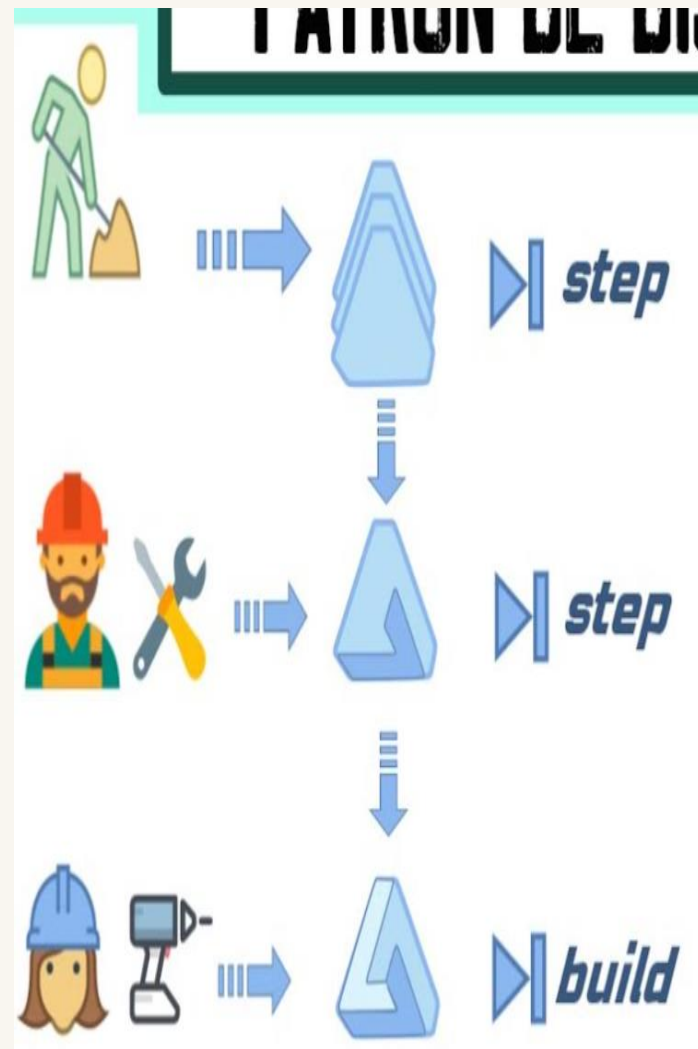
Estructuran el código de forma coherente y predecible

Mantenimiento

Facilitan la evolución del sistema sin romper lo existente

Reutilización

Soluciones probadas aplicables en múltiples contextos

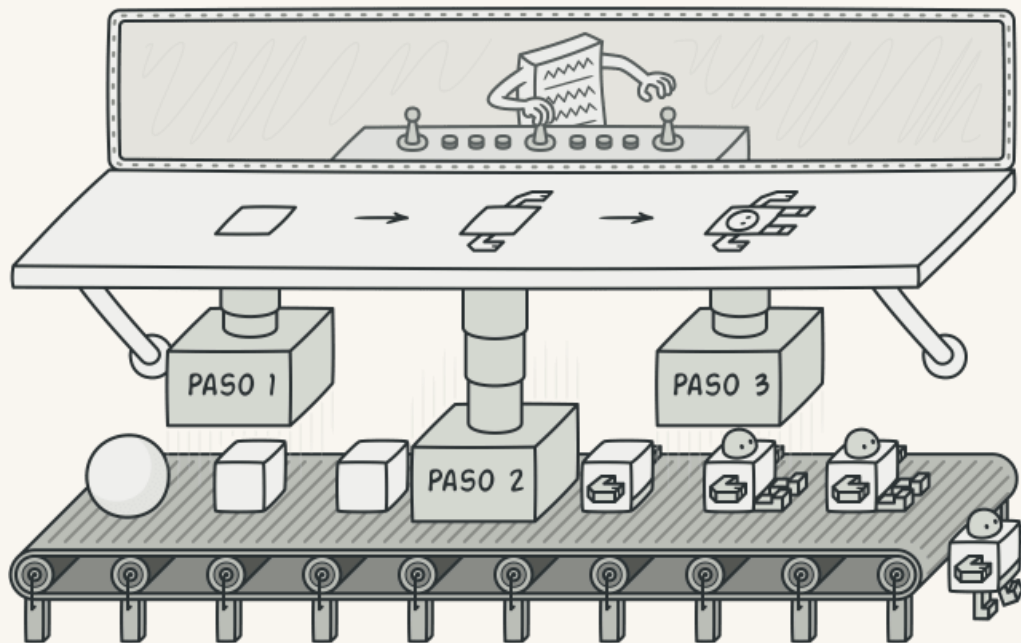


¿Qué es el patrón Builder?

Builder es un patrón **creacional** que permite construir objetos complejos **paso a paso**, separando el proceso de construcción de la representación final del objeto. En lugar de llamar a un constructor con decenas de parámetros, el cliente configura el objeto de forma incremental y legible.



Su objetivo principal es **facilitar la creación de objetos** con muchas configuraciones posibles.



El problema: constructores explosivos

Cuando una clase tiene muchos atributos, los constructores tradicionales se vuelven ilegibles y propensos a errores.

Constructor tradicional

```
new Computadora(  
    "Intel i7",  
    "32GB",  
    "1TB SSD",  
    "RTX 4060",  
    true,  
    true  
)
```

¿Qué significa cada parámetro? ¿En qué orden van?

Con Builder

```
new ComputadoraBuilder()  
    .setProcesador("Intel i7")  
    .setRAM("32GB")  
    .setSSD("1TB SSD")  
    .setGrafica("RTX 4060")  
    .setWiFi(true)  
    .build()
```

Cada paso es explícito, ordenado y fácil de entender.

Ejemplo práctico: tienda de computadoras

Una tienda necesita armar **diferentes configuraciones** para distintos tipos de clientes, reutilizando el mismo proceso de construcción.

1

Computadora básica

Para tareas cotidianas: navegación, ofimática y multimedia ligera

2

Computadora de oficina

Mayor RAM y almacenamiento para productividad empresarial

3

Computadora gamer

GPU de alto rendimiento, RGB y conectividad avanzada



Componentes de una Computadora

El patrón Builder permite configurar cada pieza de forma independiente antes de ensamblar el objeto final.



Procesador

El cerebro del sistema



Memoria RAM

Velocidad de operación



Tarjeta gráfica

Renderizado visual



Disco SSD

Almacenamiento rápido



Bluetooth

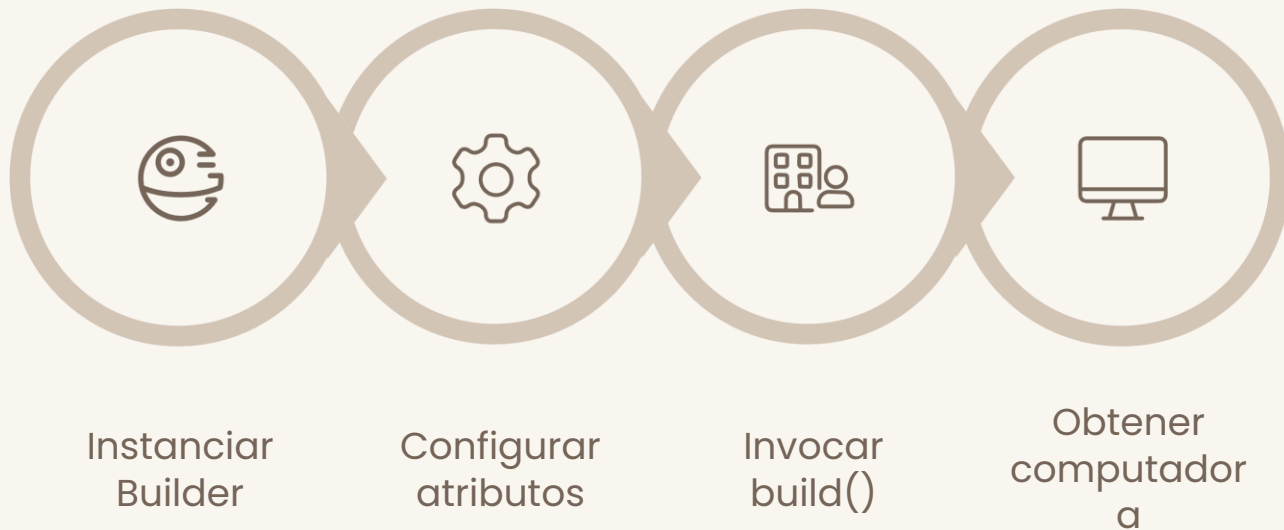
Conexión periféricos



WiFi

Conectividad inalámbrica

¿Cómo funciona el patrón?

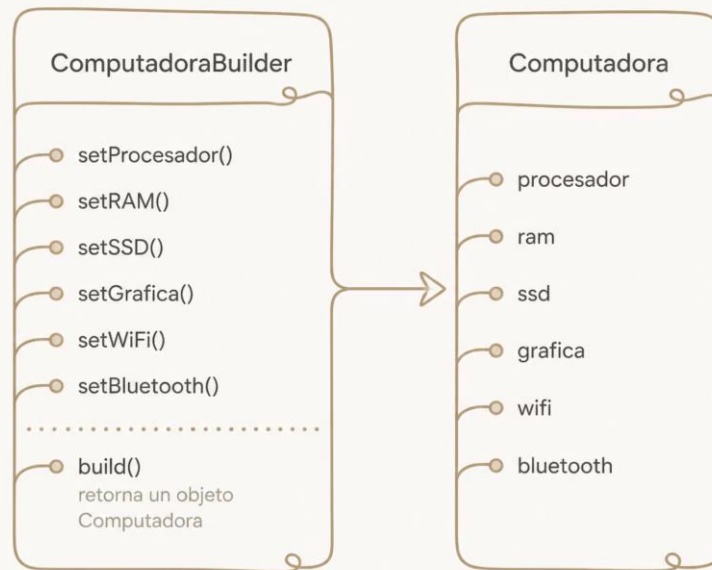


El flujo es simple y expresivo: se instancia el builder, se encadenan las configuraciones deseadas y finalmente se invoca `build()` para obtener el objeto completamente construido.

Diagrama de clases del patrón

El patrón separa dos responsabilidades claras: la **Computadora** (el producto) almacena los datos, mientras que el **ComputadoraBuilder** (el constructor) sabe cómo armarla paso a paso.

📌 El cliente nunca manipula directamente los atributos de Computadora; siempre usa el builder.

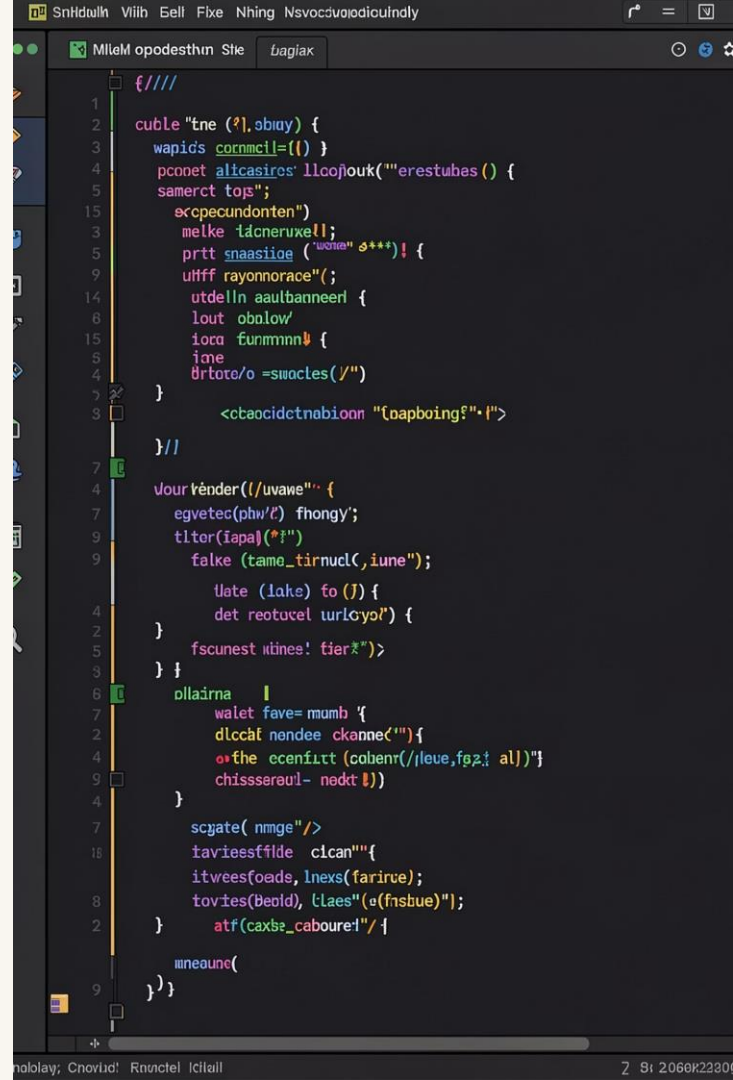


Implementación en TypeScript

La clase `ComputadoraBuilder` permite configurar cada componente mediante métodos encadenados antes de generar el objeto final.

```
const gamer = new ComputadoraBuilder()
  .setProcesador("Intel i7")
  .setRAM("32GB")
  .setSSD("1TB NVMe")
  .setGrafica("RTX 4060")
  .setWiFi(true)
  .setBluetooth(true)
  .build();
```

Cada llamada retorna el mismo builder (`this`), permitiendo el encadenamiento fluido de métodos.



Ventajas y conclusiones



Código legible

Cada parámetro tiene nombre y propósito explícito



Fácil mantenimiento

Agregar nuevos atributos no rompe el código existente



Múltiples configuraciones

Un mismo builder produce objetos muy distintos



Reutilización

El proceso de construcción se comparte entre productos

Builder es la elección ideal cuando necesitas construir objetos complejos con muchas combinaciones posibles, manteniendo el código limpio y expresivo.